

EC200U&EG91xU Series QuecOpen(SDK) **GPIO Development Guide**

LTE Standard Module Series

Version: 1.1

Date: 2023-10-24

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-10-08	JoJo YAN	Creation of the document
1.0	2021-10-21	JoJo YAN	First official release
1.1	2023-10-24	Arvin XU	1. Updated the document name based on the unified QuecOpen naming. 2. Added applicable modules: EG912U-GL and EG915U series. 3. Updated the ports in device manager for EC200U series module (Figure 1).

Contents

About the Document	3
Contents	4
Table Index	6
1 Introduction	7
2 GPIO API	8
2.1. Header File	8
2.2. API Overview	8
2.3. API Description	9
2.3.1. ql_gpio_init	9
2.3.1.1. ql_GpioNum	9
2.3.1.2. ql_GpioDir	10
2.3.1.3. ql_PullMode	11
2.3.1.4. ql_LvlMode	11
2.3.1.5. ql_errcode_gpio	12
2.3.2. ql_gpio_deinit	12
2.3.3. ql_gpio_set_level	13
2.3.4. ql_gpio_get_level	13
2.3.5. ql_gpio_set_direction	14
2.3.6. ql_gpio_get_direction	14
2.3.7. ql_gpio_set_pull	15
2.3.8. ql_gpio_get_pull	15
2.3.9. ql_int_register	15
2.3.9.1. ql_TriggerMode	16
2.3.9.2. ql_DebounceMode	17
2.3.9.3. ql_EdgeMode	17
2.3.10. ql_int_unregister	18
2.3.11. ql_int_enable	18
2.3.12. ql_int_disable	18
2.3.13. ql_pin_set_func	19
2.3.14. ql_pin_get_func	19
2.3.15. ql_pin_set_gpio	20
3 GPIO Configuration	21
3.1. Configuring GPIOs Together	21
3.2. Configuring GPIO Individually	21
4 Development Example	23
4.1. GPIO Development Example on App	23
4.1.1. GPIO Demo	23
4.2. GPIO Debugging	24
4.2.1. Debugging Preparation	24
4.2.2. GPIO Feature Debugging	25

4.2.3.	GPIO Interrupt Debugging	25
5	Appendix References	27

Table Index

Table 1: API Overview..... 8

Table 2: Related Documents..... 27

Table 3: Terms and Abbreviations..... 27

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces in detail the reference manual of GPIO in App of EC200U series and EG91xU family modules, including data structures, related APIs and debugging method.

NOTE

The GPIO number in this document refers to the GPIO number of the chip embedded in the module, not the pin number of the module. For the relationship between the GPIO number and the pin number, contact Quectel Technical Support for the GPIO configuration of individual models.

2 GPIO API

2.1. Header File

ql_gpio.h, the header file of GPIO API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are all in this directory.

2.2. API Overview

Table 1: API Overview

Function	Description
<i>ql_gpio_init()</i>	Initializes the configuration of a specified GPIO.
<i>ql_gpio_deinit()</i>	De-initializes the configured GPIO.
<i>ql_gpio_set_level()</i>	Sets the pin level of the specified GPIO.
<i>ql_gpio_get_level()</i>	Gets the pin level of the specified GPIO.
<i>ql_gpio_set_direction()</i>	Sets the pin direction of the specified GPIO.
<i>ql_gpio_get_direction()</i>	Gets the pin direction of the specified GPIO.
<i>ql_gpio_set_pull()</i>	Sets the pin pull selection of the specified GPIO.
<i>ql_gpio_get_pull()</i>	Gets the pin pull selection of the specified GPIO.
<i>ql_int_register()</i>	Configures the interrupt triggering method of the specified GPIO.
<i>ql_int_unregister()</i>	Unregisters GPIO interrupt callback function.
<i>ql_int_enable()</i>	Enables the disabled GPIO interrupt.
<i>ql_int_disable()</i>	Disables the enabled GPIO interrupt.

<code>ql_pin_set_func()</code>	Sets the pin feature.
<code>ql_pin_get_func()</code>	Obtains the feature of the pin.
<code>ql_pin_set_gpio()</code>	Sets the GPIO feature of the pin.

2.3. API Description

2.3.1. ql_gpio_init

This function initializes the configuration of a specified GPIO, including pin direction, pin pull selection, and default level of the output pin. This function can be called at its first setting or resetting of GPIO.

● Prototype

```
ql_errcode_gpio ql_gpio_init(ql_GpioNum gpio_num, ql_GpioDir gpio_dir, ql_PullMode gpio_pull,
ql_LvlMode gpio_lvl)
```

● Parameter

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_dir:

[In] Pin direction. See **Chapter 2.3.1.2** for details.

gpio_pull:

[In] Pin pull selection which is only for input pins. See **Chapter 2.3.1.3** for details.

gpio_lvl:

[In] Pin level which is only for output pins. See **Chapter 2.3.1.4** for details.

● Return Value

See **Chapter 2.3.1.5** for details.

2.3.1.1. ql_GpioNum

The enumeration of GPIO number:

```
typedef enum
{
    GPIO_0 = 0,
    GPIO_1,
```

```

GPIO_2,
GPIO_3,
GPIO_4,
GPIO_5,
.....
GPIO_28,
GPIO_29,
GPIO_30,
GPIO_31,
GPIO_MAX
}ql_GpioNum;

```

● Member

Member	Description
<i>GPIO_0</i>	GPIO number 0
<i>GPIO_1</i>	GPIO number 1
.....	
<i>GPIO_31</i>	GPIO number 31

NOTE

Contact Quectel Technical Support for GPIO configuration details on the GPIOs available on the specific model.

2.3.1.2. ql_GpioDir

The enumeration of GPIO pin directions:

```

typedef enum
{
    GPIO_INPUT,
    GPIO_OUTPUT
}ql_GpioDir;

```

● Member

Member	Description
<i>GPIO_INPUT</i>	Input

GPIO_OUTPUT

Output

2.3.1.3. qI_PullMode

Pin pull selection is only valid for input pins. The enumeration of pin pull selection:

```
typedef enum
{
    PULL_NONE,
    PULL_DOWN,
    PULL_UP
}qI_PullMode;
```

● Member

Member	Description
<i>PULL_NONE</i>	Floating
<i>PULL_DOWN</i>	Pull-down
<i>PULL_UP</i>	Pull-up

2.3.1.4. qI_LvlMode

The pin level is only valid for output pins. The enumeration of pin level:

```
typedef enum
{
    LVL_LOW,
    LVL_HIGH
}qI_LvlMode;
```

● Member

Member	Description
<i>LVL_LOW</i>	Low level
<i>LVL_HIGH</i>	High level

2.3.1.5. ql_errcode_gpio

GPIO error codes indicate whether the API is executed successfully. If it fails, it returns the error reason. The enumeration of error codes:

```
typedef enum
{
    QL_GPIO_SUCCESS = QL_SUCCESS,
    QL_GPIO_EXECUTE_ERR    = 1|QL_GPIO_ERRCODE_BASE,
    QL_GPIO_INVALID_PARAM_ERR,
    QL_GPIO_OPEN_ERR,
    QL_GPIO_CONFIG_ERR,
    QL_GPIO_CALLBACK_ERR,
    QL_GPIO_LEVEL_TRIGGER_ERR
    QL_GPIO_MEM_NULL_ERR
}ql_errcode_gpio;
```

● Member

Member	Description
<i>QL_GPIO_SUCCESS</i>	Successful execution.
<i>QL_GPIO_EXECUTE_ERR</i>	Failed execution.
<i>QL_GPIO_INVALID_PARAM_ERR</i>	Parameter is invalid.
<i>QL_GPIO_OPEN_ERR</i>	Fail to open.
<i>QL_GPIO_CONFIG_ERR</i>	Fail to configure the parameter(s).
<i>QL_GPIO_CALLBACK_ERR</i>	Fail to set the interrupt callback function.
<i>QL_GPIO_LEVEL_TRIGGER_ERR</i>	Invalid level-triggered interrupt method.
<i>QL_GPIO_MEM_NULL_ERR</i>	The pointer parameter is NULL

2.3.2. ql_gpio_deinit

This function de-initializes the configured GPIO.

● Prototype

```
ql_errcode_gpio ql_gpio_deinit(ql_GpioNum gpio_num)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.3. ql_gpio_set_level

This function sets the pin level of the specified GPIO and is valid only for output pins.

- **Prototype**

```
ql_errcode_gpio ql_gpio_set_level(ql_GpioNum gpio_num, ql_LvlMode gpio_lvl)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_lvl:

[In] Pin level which is only for output pins. See **Chapter 2.3.1.4** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.4. ql_gpio_get_level

This function gets the pin level of the specified GPIO.

- **Prototype**

```
ql_errcode_gpio ql_gpio_get_level(ql_GpioNum gpio_num, ql_LvlMode *gpio_lvl)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_lvl:

[Out] The obtained pin level. See **Chapter 2.3.1.4** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.5. ql_gpio_set_direction

This function sets the pin direction of the specified GPIO.

- **Prototype**

```
ql_errcode_gpio ql_gpio_set_direction(ql_GpioNum gpio_num, ql_GpioDir gpio_dir)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_dir:

[In] Pin direction. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.6. ql_gpio_get_direction

This function gets the pin direction of the specified GPIO.

- **Prototype**

```
ql_errcode_gpio ql_gpio_get_direction(ql_GpioNum gpio_num, ql_GpioDir *gpio_dir)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_dir:

[Out] The obtained pin direction. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.7. ql_gpio_set_pull

This function sets the pin pull selection of the specified GPIO, and it is valid only for input pins.

- **Prototype**

```
ql_errcode_gpio ql_gpio_set_pull(ql_GpioNum gpio_num, ql_PullMode gpio_pull)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_pull:

[In] Pin pull selection which is only for input pins. See **Chapter 2.3.1.3** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.8. ql_gpio_get_pull

This function gets the pin pull selection of the specified GPIO.

- **Prototype**

```
ql_errcode_gpio ql_gpio_get_pull(ql_GpioNum gpio_num, ql_PullMode *gpio_pull)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_pull:

[Out] The obtained pin pull selection. See **Chapter 2.3.1.3** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.9. ql_int_register

This function configures the interrupt triggering method of the specified GPIO. It is only valid for the input pins. This function can be used at its first setting or resetting of GPIO. You need to call *ql_int_enable()* to enable the interrupt callback function after call *ql_int_register()*.

● Prototype

```
ql_errcode_gpio ql_int_register(ql_GpioNum gpio_num,
                                ql_TriggerMode gpio_trigger, ql_DebounceMode gpio_debounce,
                                ql_EdgeMode gpio_edge, ql_PullMode gpio_pull,
                                void *int_cb, void *cb_ctx)
```

● Parameter

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

gpio_trigger:

[In] Interrupt triggering method selection. See **Chapter 2.3.9.1** for details.

gpio_debounce:

[In] Hardware debounce mode selection of the interrupt. See **Chapter 2.3.9.2** for details.

gpio_edge:

[In] Interrupt triggering method selection. See **Chapter 2.3.9.3** for details.

gpio_pull:

[In] Pin pull selection of the interrupt and is on for input pins. See **Chapter 2.3.1.3** for details.

int_cb:

[In] Interrupt callback function. This function will be called when the interrupt is triggered normally.

cb_ctx:

[In] Interrupt callback function parameter. It is used when calling the interrupt callback function.

● Return Value

See **Chapter 2.3.1.5** for details.

2.3.9.1. ql_TriggerMode

The interrupt triggering is only valid for input pins. The enumeration of interrupt triggering mode:

```
typedef enum
{
    EDGE_TRIGGER,
    LEVEL_TRIGGER
}ql_TriggerMode;
```

- Member

Member	Description
<i>EDGE_TRIGGER</i>	Edge-triggered
<i>LEVEL_TRIGGER</i>	Level-triggered

2.3.9.2. ql_DebounceMode

Interrupt triggering is only valid for input pins. The enumeration of hardware debounce modes:

```
typedef enum
{
    DEBOUNCE_DIS,
    DEBOUNCE_EN
}ql_DebounceMode;
```

- Member

Member	Description
<i>DEBOUNCE_DIS</i>	Hardware debounce is disabled.
<i>DEBOUNCE_EN</i>	Hardware debounce is enabled.

2.3.9.3. ql_EdgeMode

Interrupt triggering is only valid for input pins. The enumeration of interrupt triggering methods:

```
typedef enum
{
    EDGE_RISING,
    EDGE_FALLING,
    EDGE_BOTH
}ql_EdgeMode;
```

- Member

Member	Description
<i>EDGE_RISING</i>	Edge-triggered: Interrupt triggered at rising edge; Level-triggered: Interrupt triggered at high level.
<i>EDGE_FALLING</i>	Edge-triggered: Interrupt triggered at falling edge; Level-triggered: Interrupt triggered at low level.
<i>EDGE_BOTH</i>	Interrupt triggered at both edges.

2.3.10. ql_int_unregister

This function unregisters GPIO interrupt callback function. After calling this function, you need to re-register the interrupt callback function.

- **Prototype**

```
ql_errcode_gpio ql_int_unregister(ql_GpioNum gpio_num)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.11. ql_int_enable

This function enables the disabled GPIO interrupt. If you do not use *ql_int_unregister()* to unregister the callback function, you can call the interface again to enable the interrupt without reconfiguring the interrupt. This function needs to be used in conjunction with *ql_int_register()* to effectively interrupt the callback function.

- **Prototype**

```
ql_errcode_gpio ql_int_enable(ql_GpioNum gpio_num)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.12. ql_int_disable

This function disables the enabled GPIO interrupt. Call *ql_int_enable()* to enable the GPIO interrupt callback again.

- **Prototype**

```
ql_errcode_gpio ql_int_disable(ql_GpioNum gpio_num)
```

- **Parameter**

gpio_num:

[In] The GPIO number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.13. ql_pin_set_func

This function sets the pin feature.

- **Prototype**

```
ql_errcode_gpio ql_pin_set_func(uint8_t pin_num, uint8_t func_sel)
```

- **Parameter**

pin_num:

[In] The pin number.

func_sel:

[In] The function number of the pin.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.14. ql_pin_get_func

This function obtains the feature of the pin.

- **Prototype**

```
ql_errcode_gpio ql_pin_get_func(uint8_t pin_num, uint8_t *func_sel)
```

- **Parameter**

pin_num:

[In] The pin number.

func_sel:

[Out] The function number of the pin.

- **Return Value**

See **Chapter 2.3.1.5** for details.

2.3.15. ql_pin_set_gpio

This function sets the GPIO feature of the pin.

- **Prototype**

```
ql_errcode_gpio ql_pin_set_gpio(uint8_t pin_num)
```

- **Parameter**

pin_num:

[In] The pin number.

- **Return Value**

See **Chapter 2.3.1.5** for details.

3 GPIO Configuration

This chapter introduces how to use the above API to configure GPIO.

3.1. Configuring GPIOs Together

Step 1: Establish GPIO parameter setting table as below.

```
static ql_gpio_cfg_t ql_gpio_cfg[] =
{
    /* gpio_num gpio_dir gpio_pull gpio_lvl */
    { GPIO_0, GPIO_INPUT, PULL_DOWN, 0xff }, // set input pull-down
    { GPIO_1, GPIO_OUTPUT, 0xff, LVL_HIGH } // set output high-level
};
```

Step 2: Use the initialization function similar to *ql_gpio_demo_init()* to quickly configure the parameters of multiple pins. The usage is shown in the figure below.

```
void _ql_gpio_demo_init(void)
{
    uint16_t num;
    for (num = 0; num < sizeof(_ql_gpio_cfg)/sizeof(_ql_gpio_cfg[0]); num++)
    {
        ql_gpio_deinit(_ql_gpio_cfg[num].gpio_num);
        ql_gpio_init(_ql_gpio_cfg[num].gpio_num, _ql_gpio_cfg[num].gpio_dir, _ql_gpio_cfg[num].gpio_pull, _ql_gpio_cfg[num].gpio_lvl);
    }
}
```

3.2. Configuring GPIO Individually

Step 1: To set GPIO as an input pin, you need to set the GPIO number, pin direction, and pin pull selection.

```
/* set input pull-up */
ql_gpio_set_direction(GPIO_0, GPIO_INPUT);
ql_gpio_set_pull(GPIO_0, PULL_UP);
```

Step 2: To set GPIO as an output pin, you need to set the GPIO number, pin direction, and output pin level.

```

/*set output high*/
ql_gpio_set_direction(GPIO_0, GPIO_OUTPUT);
ql_gpio_set_level(GPIO_0, LVL_HIGH);

```

Step 3: To set GPIO as interrupt triggering, you need to set GPIO number, triggering mode, debounce selection, triggering method, pin pull selection and interrupt callback function, as shown in the figure below.

```

ql_pin_set_func(QL_TEST1_PIN_GPIO0, QL_TEST1_PIN_GPIO0_FUNC_GPIO); .....//set GPIO0
ql_pin_set_func(QL_TEST1_PIN_GPIO1, QL_TEST1_PIN_GPIO1_FUNC_GPIO); .....//set GPIO1

//ql_gpio_deinit(GPIO_0);
//ql_gpio_deinit(GPIO_1);

ql_int_register(GPIO_0, EDGE_TRIGGER, DEBOUNCE_EN, EDGE_RISING, PULL_DOWN, _gpioint_callback01, NULL);
ql_int_register(GPIO_1, EDGE_TRIGGER, DEBOUNCE_EN, EDGE_FALLING, PULL_UP, _gpioint_callback02, NULL);

ql_int_enable(GPIO_0);
ql_int_enable(GPIO_1);

```

NOTE

This configuration requires calling *ql_int_enable()* to enable interrupts.

4 Development Example

This chapter introduces how to use the above API for GPIO development and simple debugging on the App. In the GPIO example, GPIO0 and GPIO1 are used by default for testing.

4.1. GPIO Development Example on App

4.1.1. GPIO Demo

The module QuecOpen CSDK provides GPIO examples, namely *gpio_demo.c* and *gpio_int_demo.c* files.

- The *gpio_demo.c* file contains the basic settings for GPIOs, including pin direction, pin level, and pin pull selection. The entry function is *ql_gpio_app_init()*, as shown in the demo below.

For example, both pin a and pin b can be multiplexed to GPIO0, but they cannot be configured as GPIO0 at the same time. The demo changes the GPIO configuration at a frequency of every 3 seconds. In other words, the LED light is on for 3 seconds and off for 3 seconds. After one round of execution is completed, switch to other pins with the same GPIO name, and execute GPIO settings repeatedly.

```
void ql_gpio_app_init(void)
{
    ... QLOSStatus.err = QL_OSI_SUCCESS;
    ... ql_task_t.gpio_task = NULL;

    ... err = ql_rtos_task_create(&gpio_task, 1024, APP_PRIORITY_NORMAL, "ql_gpiodemo", ql_gpio_demo_thread, NULL, 1);
    ... if (err != QL_OSI_SUCCESS)
    ... {
    ...     QL_GPIODEMO_LOG("gpio_demo_task_created_failed");
    ... }
}
```

- The *gpio_int_demo.c* file contains the basic settings for GPIO interrupt, including interrupt callback function, interrupt triggering mode. The entry function is *ql_gpioint_app_init()*, as shown in the demo below.

At the beginning of the demo, both GPIO0 and GPIO1 can trigger interrupts. After that, in every 20 seconds, only one GPIO can trigger the interrupts. GPIO0 and GPIO1 loops are valid.

```
void ql_gpioint_app_init(void)
{
    ... QLOSStatus.err = QL_OSI_SUCCESS;
    ... ql_task_t.gpioint_task = NULL;

    ... err = ql_rtos_task_create(&gpioint_task, 1024, APP_PRIORITY_NORMAL, "ql_gpiointdemo", ql_gpioint_demo_thread, NULL, 1);
    ... if (err != QL_OSI_SUCCESS)
    ... {
    ...     QL_GPIOINTDEMO_LOG("gpio_int_demo_task_created_failed");
    ... }
}
```


NOTE

The above demos are disabled by default in the `ql_init_demo_thread()`, as shown in the figure below.

```
static void ql_init_demo_thread(void *param)
{
    ...QL_INIT_LOG("init demo thread enter, param 0x%x", param);
    #if 0
    ...ql_gpio_app_init();
    ...ql_gpioint_app_init();
    #endif

    #ifdef QL_APP_FEATURE_LEDCFG
    ...ql_ledcfg_app_init();
    #endif

    #ifdef QL_APP_FEATURE_AUDIO
    ...//ql_audio_app_init();
    #endif
    #ifdef QL_APP_FEATURE_LCD
    ...//ql_lcd_app_init();
    #endif
    ...//ql_nw_app_init();
    ...//ql_datacall_app_init();
    >> //ql_osi_demo_init();
}
```

4.2. GPIO Debugging

4.2.1. Debugging Preparation

GPIO feature of the module can be debugged on Quectel LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to a PC with a USB cable. The USB AP Log Port mainly displays the system debugging information. You can view information of the example through USB AP Log Port after capturing the log with cooltools. For details about log capture, see **document [2]**.

-  Quectel USB AP Log Port (COM93)
-  Quectel USB AT Port (COM92)
-  Quectel USB CP Log Port (COM95)
-  Quectel USB Diag Port (COM91)
-  Quectel USB MOS Port (COM94)
-  Quectel USB NMEA Port (COM113)

Figure 1: Ports in Device Manager

4.2.2. GPIO Feature Debugging

ql_gpio_app_init() runs automatically after the module is booted. Through the logs, you can see that GPIO0 and GPIO1 are cyclically set as output and input, and the pin level changes cyclically. You can also use a multimeter or oscilloscope to measure the corresponding pins to check whether the settings are correct.

Debugging information printed from debug UART port is shown in the figure below:

	Description
[ql_GPIODEMO][ql_gpio_demo_thread, 189]	GPIO0/1 pin is TEST1 -> TEST2
[ql_GPIODEMO][ql_gpio_demo_thread, 103]	gpio[0] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 104]	gpio[0] set dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 110]	gpio[0] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 111]	gpio[0] get dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 103]	gpio[1] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 104]	gpio[1] set dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 110]	gpio[1] output low-level
[ql_GPIODEMO][ql_gpio_demo_thread, 111]	gpio[1] get dir:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 124]	gpio[0] output high-level
[ql_GPIODEMO][ql_gpio_demo_thread, 125]	gpio[0] set dir:[1], lvl:[1]
[ql_GPIODEMO][ql_gpio_demo_thread, 131]	gpio[0] output high-level
[ql_GPIODEMO][ql_gpio_demo_thread, 132]	gpio[0] get dir:[1], lvl:[1]
[ql_GPIODEMO][ql_gpio_demo_thread, 124]	gpio[1] output high-level
[ql_GPIODEMO][ql_gpio_demo_thread, 125]	gpio[1] set dir:[1], lvl:[1]
[ql_GPIODEMO][ql_gpio_demo_thread, 131]	gpio[1] output high-level
[ql_GPIODEMO][ql_gpio_demo_thread, 132]	gpio[1] get dir:[1], lvl:[1]
[ql_GPIODEMO][ql_gpio_demo_thread, 145]	gpio[0] input pull-down
[ql_GPIODEMO][ql_gpio_demo_thread, 146]	gpio[0] set dir:[0], pull:[1]
[ql_GPIODEMO][ql_gpio_demo_thread, 153]	gpio[0] input pull-down
[ql_GPIODEMO][ql_gpio_demo_thread, 154]	gpio[0] get dir:[0], pull:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 145]	gpio[1] input pull-down
[ql_GPIODEMO][ql_gpio_demo_thread, 146]	gpio[1] set dir:[0], pull:[1]
[ql_GPIODEMO][ql_gpio_demo_thread, 153]	gpio[1] input pull-down
[ql_GPIODEMO][ql_gpio_demo_thread, 154]	gpio[1] get dir:[0], pull:[1], lvl:[0]
[ql_GPIODEMO][ql_gpio_demo_thread, 167]	gpio[0] input pull-up

4.2.3. GPIO Interrupt Debugging

ql_gpioint_app_init() runs automatically after the module is booted, and GPIO0 and GPIO1 are set as interrupts. Please manually trigger the corresponding pins (connect to 1.8 V power supply or ground wire), then you can see that the interrupt callback is triggered. Print the related logs. Both GPIOs can trigger the interrupt in the first 20 seconds, and only one GPIO triggers an interrupt every 20 seconds thereafter after that.

Debugging information printed from debug UART port is shown in the figure below:

GPIO int status 0x20000000
GPIO int status 0x20000000
GPIO int status 0x20000000
GPIO int status 0x20000000
GPIO int status 0x20000000
GPIO int status 0x20000000
GPIO int status 0x40000000
GPIO int status 0x40000000
GPIO int status 0x40000000
GPIO int status 0x40000000
GPIO int status 0x40000000
GPIO int status 0x40000000
GPIO int status 0x40000000

NOTE

1. Because the demos are closed by default, please turn on the preprocessing before the test.
2. Because the pins in this demo may also be configured in other demos, only this demo is used for testing.

5 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
AP	Application Processor
EVB	Evaluation Board
GPIO	General-Purpose Input/Output
IoT	Internet of Things
PC	Personal Computer
RTOS	Real-Time Operating System
SDK	Software Development Kit
USB	Universal Serial Bus
UART	Universal Asynchronous Receiver/Transmitter